

# Домашнее задание 4. Изучение современных инструментов КОМПОЗИЦИОННОГО анализа кода

## Contents

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Disclaimer.....                                                         | 2  |
| Подготовка к выполнению задания .....                                   | 2  |
| Анализ с помощью Dependency Check .....                                 | 2  |
| Правка build.gradle.kts .....                                           | 2  |
| Получаем NVD API Key для ускорения загрузки баз уязвимостей .....       | 3  |
| Строим отчет с помощью dependency check.....                            | 3  |
| Скриншоты отчета Dependency Check .....                                 | 3  |
| Настройка OWASP Dependency Track.....                                   | 4  |
| Настраиваем CycloneDX.....                                              | 4  |
| Правка build.gradle.kts .....                                           | 4  |
| Генерация SBOM.....                                                     | 4  |
| Подготовка файла анализа SBOM.....                                      | 5  |
| Валидация SBOM .....                                                    | 5  |
| Количество компонентов .....                                            | 5  |
| Анализ SBOM-файла (CycloneDX, bom.json) .....                           | 6  |
| Настройка Dependency Track.....                                         | 7  |
| Ставим docker и docker compose.....                                     | 7  |
| Создаем проект нашего приложения для Dependency Track.....              | 8  |
| Подключаем OSS Index .....                                              | 8  |
| Загружаем полученный ранее BOM-файл в Dependency Track.....             | 9  |
| Анализируем результаты с помощью web .....                              | 10 |
| Выгрузка результатов.....                                               | 11 |
| Сравнительный анализ результатов.....                                   | 11 |
| Результаты Dependency-Check (отчёт: dependency-check-report.html) ..... | 11 |
| Результаты Dependency-Track (экспорт: dtrack-findings.json) .....       | 11 |

|                                                            |    |
|------------------------------------------------------------|----|
| Уязвимости, найденные только одним инструментом .....      | 12 |
| Объяснение разницы между инструментами .....               | 12 |
| Сравнение по критериям из задания .....                    | 12 |
| Установка и настройка CI/CD на GitHub.....                 | 13 |
| Подключаем self hosted github runner.....                  | 13 |
| Проверка работоспособности github self hosted runner ..... | 15 |
| Настройка секретов GitHub .....                            | 16 |
| Создаем файл CI/CD в Github Actions .....                  | 16 |
| Результат запуска .....                                    | 20 |

## Disclaimer

В отчете осознанно показываются ключи. Я понимаю, что в Production-варианте ключи нельзя вставлять в документы и надо хранить безопасно.

Я храню проекты в GitHub, поэтому я использую GitHub Actions , а не Gitlab. Легитимное использование GitHub подтверждено в чате группы курса.

## Подготовка к выполнению задания

Используем проект Java Gradle Kotlin DSL - shortlinksapp. Код размещен в репозитории [vkuryndin/shortlinkapp: initial commit](https://github.com/vkuryndin/shortlinkapp)

## Анализ с помощью Dependency Check

Dependency-Check запускается локально через Gradle-плагин с формированием HTML-отчёта.

Добавляем плагины в наш проект shortlinksapp.

### Правка build.gradle.kts

```
import org.owasp.dependencycheck.gradle.extension.DependencyCheckExtension
import org.owasp.dependencycheck.reporting.ReportGenerator

plugins {
    id("org.owasp.dependencycheck") version "12.1.9"
}

val depCheckOutDir = layout.buildDirectory.dir("reports/dependency-check")

configure<DependencyCheckExtension> {
    // report format }
    format = ReportGenerator.Format.ALL.toString()

    // setting putput directory
    setOutputDirectory(depCheckOutDir.get().asFile.absolutePath)

    //to speed up vulnerabilities databases download process
```

```
nvd.apiKey = System.getenv("NVD_API_KEY")
}
```

## Получаем NVD API Key для ускорения загрузки баз уязвимостей

### Request an API Key

To request an NVD API Key, please provide your organization name and a valid email address, and indicate your organization type. You must scroll to end of the Terms of Use Agreement and check "I agree to the Terms of Use" to obtain an API Key. Upon submitting the request, you will receive an email containing a single-use hyperlink that is used to activate and view your API Key. If your key is not activated within seven days, a new request for an API Key must be submitted.

**Organization Name:**

Vladimir Kuryndin

**Email Address:**

vkuryndin@gmail.com

**Organization Type:**

Personal Use / Not Listed

The API is provided "as is" and on an "as-available" basis. The NVD hereby disclaim all warranties of any kind, express or implied, including without limitation the warranties of merchantability, fitness for a particular purpose, and non-infringement. The NVD makes no warranty that the API will be error free or that access thereto will be continuous or uninterrupted.

**No Waiver**  
The NVD's failure to exercise or enforce any right or provision of these Terms of Use shall not constitute waiver of such right or provision.

## Строим отчет с помощью dependency check

```
./gradlew dependencyCheckAnalyze -info
```

## Скриншоты отчета Dependency Check



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help](#): [github issues](#)

### Project: root project 'shortlinkapp'

org.example:shortlinkapp:1.0-SNAPSHOT

Scan Information ([show all](#)):

- dependency-check version: 12.1.9
- Report Generated On: Wed, 24 Dec 2025 18:35:08 +0300
- Dependencies Scanned: 65 (65 unique)
- Vulnerable Dependencies: 2
- Vulnerabilities Found: 2
- Vulnerabilities Suppressed: 0
- ...

### Summary

Summary of Vulnerable Dependencies ([click to show all](#))

| Dependency                                  | Vulnerability IDs                                              | Package                                             | Highest Severity | CVE Count | Confidence | Evidence Count |
|---------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------|------------------|-----------|------------|----------------|
| <a href="#">commons-beanutils-1.9.4.jar</a> | <a href="#">cpe:2.3:a:apache:commons-beanutils:1.9.4:*****</a> | pkg.maven/commons-beanutils/commons-beanutils@1.9.4 | HIGH             | 1         | Highest    | 171            |
| <a href="#">commons-lang3-3.8.1.jar</a>     | <a href="#">cpe:2.3:a:apache:commons-lang:3.8.1:*****</a>      | pkg.maven/org.apache.commons/commons-lang3@3.8.1    | MEDIUM           | 1         | Highest    | 143            |

### Dependencies (vulnerable)

commons-beanutils-1.9.4.jar

commons-beanutils-1.9.4.jar

Description:

Apache Commons BeanUtils provides an easy-to-use but flexible wrapper around reflection and introspection.

License:

<https://www.apache.org/licenses/LICENSE-2.0.txt>

File Path:

C:\Users\kurindin\gradle\caches\modules-2\files-2.1\commons-beanutils\commons-beanutils\1.9.4\d52b9abcd97f38c81342bb7e7ae1eee9b73cba51\commons-beanutils-1.9.4.jar

MD5:

07dc532ee316fe1f2f0323e9bd2f8df4

SHA1:

d52b9abcd97f38c81342bb7e7ae1eee9b73cba51

SHA256:

7d938c81789028045c08c065e94be75fc280527620d5bd62b519d5838532368a

Referenced In Project/Scope:

shortlinkapp:checkstyle

Included by:

pkg:maven/com.puppycrawl.tools/checkstyle@10.17.0

Evidence

Identifiers

- pkg:maven/commons-beanutils/commons-beanutils@1.9.4 (Confidence:High)
- cpe:2.3:a:apache:commons-beanutils:1.9.4:\*:\*:\*:\*:\* (Confidence:Highest) suppress

Published Vulnerabilities

CVE-2025-48734 suppress

Improper Access Control vulnerability in Apache Commons.

A special BeanIntrospector class was added in version 1.9.2. This can be used to stop attackers from using the declared class property of Java enum objects to get access to the classloader. However this protection was not enabled by default. PropertyUtilsBean (and consequently BeanUtilsBean) now disallows declared class level property access by default.

Также данный файл будет добавлен к проекту при сдаче домашнего задания (см файл homework4.zip).

## Настройка OWASP Dependency Track

Dependency-Track анализировал зависимости через загрузку SBOM (CycloneDX) и последующий анализ через подключённый OSS Index.

### Настраиваем CycloneDX

Правка build.gradle.kts

Добавляем в файл build.gradle.kts

```
plugins {  
    id("org.cyclonedx.bom") version "2.4.1"  
}
```

Никаких дополнительных настроек не делаем, так как у нас Gradle 8.x и для него поддерживается только версия cyclone dx 2.4.1

### Генерация SBOM

```
./gradlew cyclonedxBom
```

Получаем файлы bom.xml и bom.json. Данные файлы будут добавлены к проекту при сдаче домашнего задания (см файл homework4.zip)

```

<?xml version="1.0" encoding="UTF-8"?>
<bom serialNumber="urn:uuid:9fced05c-90fc-4fed-9bec-881e9a47f9c5" version="1"
xmlns="http://cyclonedx.org/schema/bom/1.6">
  <metadata>
    <timestamp>2025-12-24T14:07:07Z</timestamp>
    <tools>
      <components>
        <component type="application">
          <author>CycloneDX</author>
          <name>cyclonedx-gradle-plugin</name>
          <version>2.4.1</version>
        </component>
      </components>
    </tools>
    <component type="library" bom-ref="pkg:maven/org.example/shortlinkapp@1.0-SNAPSHOT?project_path=%3A">
      <group>org.example</group>
      <name>shortlinkapp</name>
      <version>1.0-SNAPSHOT</version>
      <purl>pkg:maven/org.example/shortlinkapp@1.0-SNAPSHOT?project_path=%3A</purl>
      <modified>false</modified>
      <externalReferences>
        <reference type="vcs">
          <url>https://github.com/vkuryndin/shortlinkapp.git</url>
        </reference>
      </externalReferences>
    </component>
  </metadata>
  <components>
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
    <component type="library" ...
  </components>
</bom>

```

## Подготовка файла анализа SBOM

Ставим cyclonedx-win-x64.exe с [Releases · CycloneDX/cyclonedx-cli](#)

```
cyclonedx analyze --input-file .\bom.xml --output-format text | Out-File -FilePath .\sbom-analysis.txt -Encoding utf8
```

Получаем файл:

```

Analysis results for shortlinkapp@1.0-SNAPSHOT:
BOM Serial Number: urn:uuid:9fced05c-90fc-4fed-9bec-881e9a47f9c5
BOM Version: 1
Timestamp: 24.12.2025 14:07:07

```

## Валидация SBOM

```

PS C:\Users\kurindinv\IdeaProjects\shortlinkapp\build\reports> cyclonedx validate --input-file .\bom.xml
BOM validated successfully.

```

## Количество компонентов

Смотрим количество компонентов

```
PS C:\Users\kurindinv\IdeaProjects\shortlinkapp\build\reports> Select-String -Path .\bom.xml -Pattern "<component " | Measure-Object

Count      : 82
Average    :
Sum        :
Maximum    :
Minimum    :
Property   :
```

## Анализ SBOM-файла (CycloneDX, bom.json)

### Скриншот

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <bom serialNumber="urn:uuid:9fced05c-90fc-4fed-9bec-881e9a47f9c5" version="1" xmlns="http://cyclonedx.org/schema/bom/1.6">
3   <metadata>
4     <timestamp>2025-12-24T14:07:07Z</timestamp>
5     <tools>
6       <components>
7         <component type="application">
8           <author>CycloneDX</author>
9           <name>cyclonedx-gradle-plugin</name>
10          <version>2.4.1</version>
11        </component>
12      </components>
13    </tools>
14    <component type="library" bom-ref="pkg:maven/org.example/shortlinkapp@1.0-SNAPSHOT?project_path=%3A">
15      <group>org.example</group>
16      <name>shortlinkapp</name>
17      <version>1.0-SNAPSHOT</version>
18      <purl>pkg:maven/org.example/shortlinkapp@1.0-SNAPSHOT?project_path=%3A</purl>
19      <modified>>false</modified>
20      <externalReferences>
21        <reference type="vcs">
22          <url>https://github.com/vkuryndin/shortlinkapp.git</url>
23        </reference>
24      </externalReferences>
25    </component>
26  </metadata>
27  <components>
28    <component type="library" ...
29    <component type="library" ...
30    <component type="library" ...
31    <component type="library" ...
32    <component type="library" ...
33    <component type="library" ...
34    <component type="library" ...
35    <component type="library" ...
36    <component type="library" ...
37    <component type="library" ...
38    <component type="library" ...
39    <component type="library" ...
40    <component type="library" ...
41    <component type="library" ...
42    <component type="library" ...
43    <component type="library" ...
44    <component type="library" ...
45    <component type="library" ...
46    <component type="library" ...
47    <component type="library" ...
48    <component type="library" ...
49    <component type="library" ...
50    <component type="library" ...
51    <component type="library" ...
52    <component type="library" ...
53    <component type="library" ...
54    <component type="library" ...
55    <component type="library" ...
56    <component type="library" ...
57    <component type="library" ...
58    <component type="library" ...
59    <component type="library" ...
60    <component type="library" ...
61    <component type="library" ...
62    <component type="library" ...
63    <component type="library" ...
64    <component type="library" ...
65    <component type="library" ...
66    <component type="library" ...
67    <component type="library" ...
68    <component type="library" ...
69    <component type="library" ...
70    <component type="library" ...
71    <component type="library" ...
72    <component type="library" ...
73    <component type="library" ...
74    <component type="library" ...
75    <component type="library" ...
76    <component type="library" ...
77    <component type="library" ...
78    <component type="library" ...
79    <component type="library" ...
80    <component type="library" ...
81    <component type="library" ...
82    <component type="library" ...
83    <component type="library" ...
84    <component type="library" ...
85    <component type="library" ...
86    <component type="library" ...
87    <component type="library" ...
88    <component type="library" ...
89    <component type="library" ...
90    <component type="library" ...
91    <component type="library" ...
92    <component type="library" ...
93    <component type="library" ...
94    <component type="library" ...
95    <component type="library" ...
96    <component type="library" ...
97    <component type="library" ...
98    <component type="library" ...
99    <component type="library" ...
100   <component type="library" bom-ref="pkg:maven/javen/javen@2.0.0?type=jar">
101     <group>javen</group>
102     <name>javen</name>
103     <version>2.0.0</version>
104     <description>javen is a universal XPath engine for Java</description>
105   </component>
106 </components>
107 </bom>
```

### Общая информация

Формат: **CycloneDX**, specVersion = 1.6, формат **JSON**, version = 1.

Дата создания: **2025-12-24T14:07:07Z**.

С помощью чего создано: **cyclonedx-gradle-plugin 2.4.1**.

Корневой компонент (сам проект): **org.example:shortlinkapp:1.0-SNAPSHOT** (group:name:version, purl), указан VCS (GitHub).

### Состав зависимостей (components):

В секции **components** перечислено **80** компонентов (все с **type = library**).

Дополнительно в **metadata** присутствуют ещё 2 компонента:

1. сам проект (`metadata.component`),
2. инструмент генерации (`metadata.tools`).

Поэтому суммарно “визуально” получается **82** компонента — это совпадает с подсчётом по `bom.xml` через поиск `<component ...>`.

Разделение на “основные/тестовые” зависимости сделано через `property`

`cdx:maven:package:test:`

- **test = true:** 20 компонентов (JUnit/Mockito и связанные библиотеки).
- **test = false:** 60 компонентов (остальные зависимости и плагины/инструменты сборки).

*Качество данных в SBOM (для дальнейшего анализа уязвимостей):*

Для всех компонентов указан **purl** (удобно для сопоставления с базами уязвимостей).

Почти везде присутствуют **hashes** (контроль целостности артефактов).

У всех компонентов заполнены **licenses** (как минимум идентификаторы лицензий).

*Примеры зависимостей, которые видно в SBOM (group:name:version):*

```
com.google.code.gson:gson:2.10.1
org.slf4j:slf4j-simple:2.0.17
com.puppycrawl.tools:checkstyle:10.17.0
org.mockito:mockito-core:5.14.2 (test)
```

*Особенности, на которые стоит обратить внимание:*

В SBOM встречаются **несколько версий одной и той же библиотеки** (в моём файле таких случаев **12**). Примеры:

- `gson:` **2.10.1** и **2.13.1**
- `commons-lang3:` **3.8.1** и **3.18.0**
- `junit-jupiter` и `related:` **5.10.0** и **5.11.2**

Это нормальная ситуация для Java-проектов: часть версий приходит транзитивно и/или из разных конфигураций (`main/test/build`). Для анализа уязвимостей важно учитывать **все реально присутствующие версии**, потому что CVE могут относиться к конкретной версии.

*Вывод*

SBOM сформирован корректно и содержит достаточно информации (`purl`, версии, лицензии, хэши и граф зависимостей)

## Настройка Dependency Track

Ставим `docker` и `docker compose`

Докер уже стоял ранее, доставили только `docker compose`.

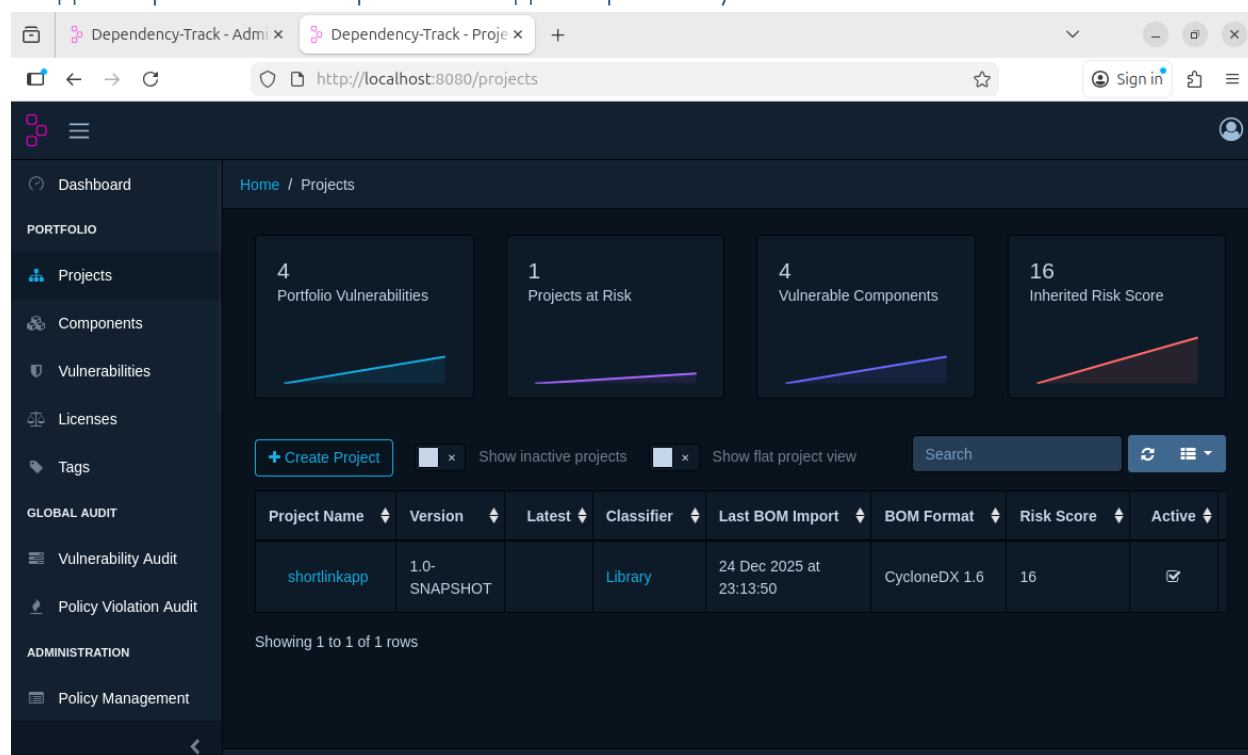
```
mkdir -p ~/dtrack && cd ~/dtrack
curl -LO https://dependencytrack.org/docker-compose.yml
docker compose up -d
```

проверяем статус

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ cd ~/dtrack
sudo docker compose up -d
sudo docker compose ps
[+] Running 6/6
 ✓ Network dtrack_default          Created                                0.0s
 ✓ Volume "dtrack_dtrack-data"     Created                                0.0s
 ✓ Volume "dtrack_postgres-data"   Created                                0.0s
 ✓ Container dtrack-postgres-1     Healthy                               5.8s
 ✓ Container dtrack-apiserver-1    Healthy                               19.5s
 ✓ Container dtrack-frontend-1     Started                              19.7s
```

| NAME               | IMAGE                     | COMMAND                  | SERVICE   | CREATED        | STATUS                  | PORTS                                       |
|--------------------|---------------------------|--------------------------|-----------|----------------|-------------------------|---------------------------------------------|
| dtrack-apiserver-1 | dependencytrack/apiserver | "/bin/sh -c 'exec ja..." | apiserver | 19 seconds ago | Up 14 seconds (healthy) | 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp |
| dtrack-frontend-1  | dependencytrack/frontend  | "/docker-entrypoint..."  | frontend  | 19 seconds ago | Up Less than a second   | 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp |
| dtrack-postgres-1  | postgres:17-alpine        | "docker-entrypoint.s..." | postgres  | 19 seconds ago | Up 19 seconds (healthy) | 5432/tcp                                    |

Создаем проект нашего приложения для Dependency Track

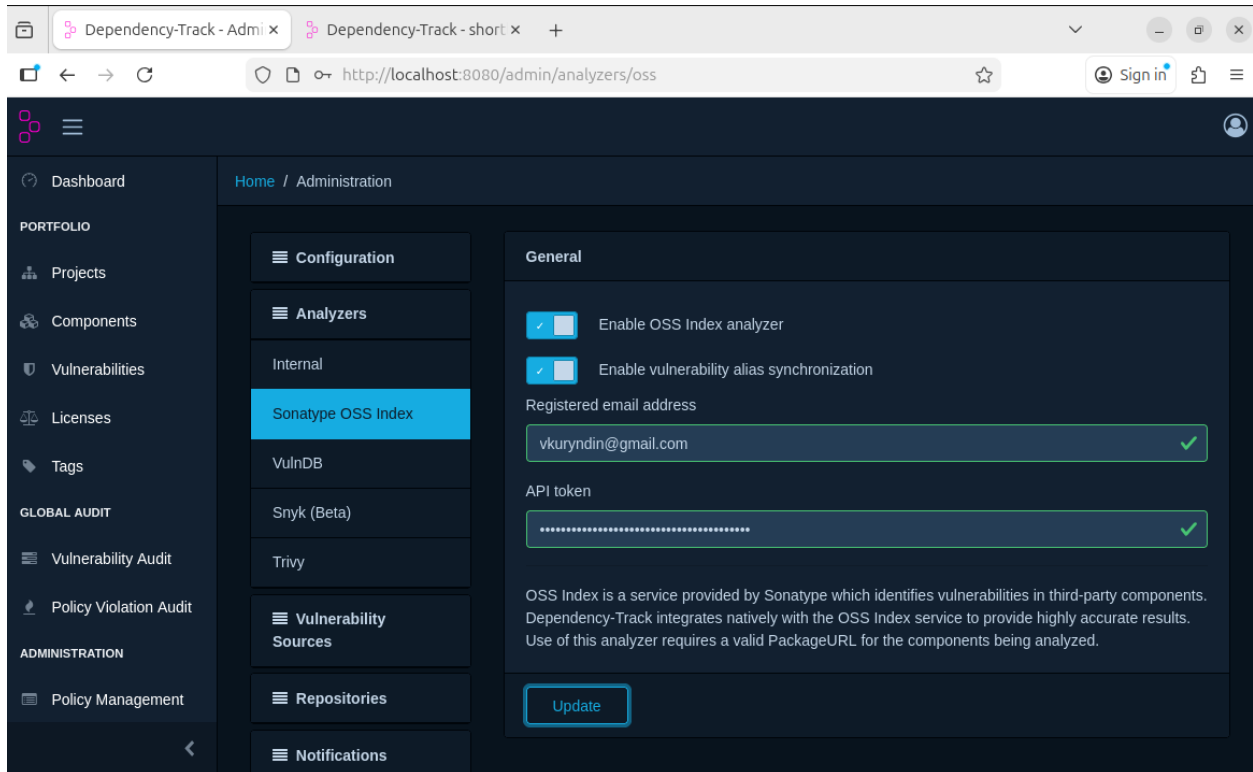


Создаем новый проект Shortlinkapp с версией 1.0-SNAPSHOT, получаем API Key через панель администрирования.

Подключаем OSS Index

Для улучшения качества работы с уязвимостям подключаем Sonatype OSS index.



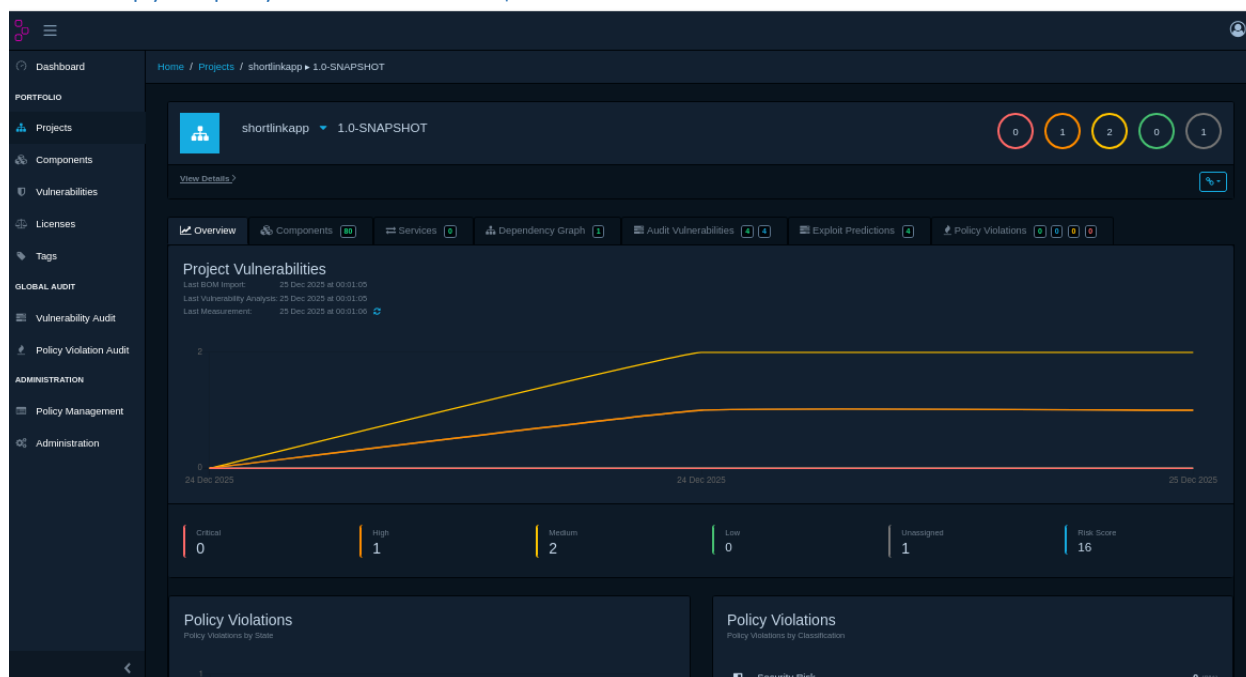


## Загружаем полученный ранее BOM-файл в Dependency Track

```
vladimr-kuryndin@vladimr-kuryndin-VMware-Virtual-Platform:~/dtrack$ sudo curl -i -X POST "http://localhost:8081/api/v1/bom" -H "X-API-Key: odt_pchp6V6x_6mIzaRhqcRdhJeZduP2K7iy8lsVKl371" -F "projectName=shortlinkapp" -F "projectVersion=1.0-SNAPSHOT" -F "autoCreate=true" -F "bon=@/mnt/hgfs/bom.xml"
HTTP/1.1 100 Continue

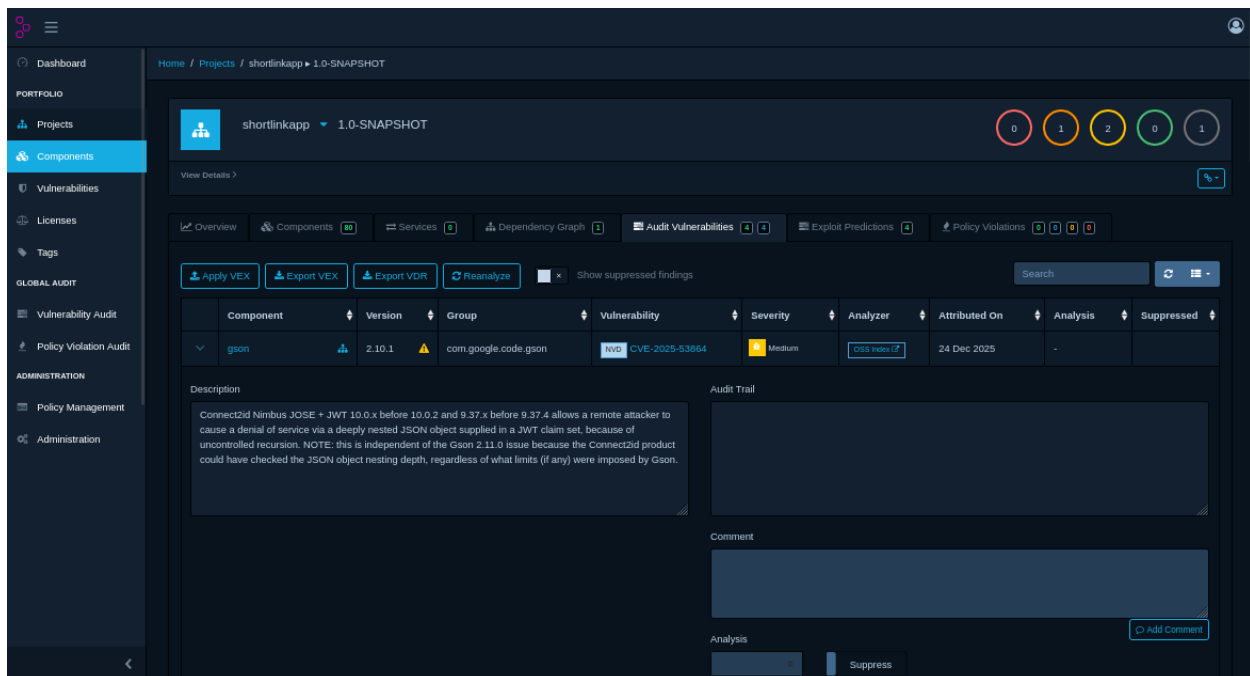
HTTP/1.1 200 OK
Date: Wed, 24 Dec 2025 20:01:05 GMT
Content-Type: application/json
X-Powered-By: Dependency-Track v4.13.6
Cache-Control: private, max-age=0, must-revalidate, no-cache
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Origin, Content-Type, Authorization, X-Requested-With, Content-Length, Accept, Origin, X-API-Key, X-Total-Count, *
Access-Control-Expose-Headers: Origin, Content-Type, Authorization, X-Requested-With, Content-Length, Accept, Origin, X-API-Key, X-Total-Count
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 3600
X-Request-Id: f7085410-ca4e-45c2-a1cd-7d285137fbd4
Content-Length: 48
```

## Анализируем результаты с помощью web



The screenshot displays the 'Components' table for the 'shortlinkapp 1.0-SNAPSHOT' project. The table lists various components with their versions, groups, licenses, risk scores, and vulnerability counts. The table is filtered to show 'Outdated only' components.

| Component       | Version | Group           | Internal | License      | Risk Score | Vulnerabilities |
|-----------------|---------|-----------------|----------|--------------|------------|-----------------|
| Saxon-HE        | 12.8    | net.sf.saxon    |          | MPL-2.0      | 0          | 0               |
| Saxon-HE        | 12.4    | net.sf.saxon    |          | MPL-2.0      | 0          | 0               |
| antlr4-runtime  | 4.13.1  | org.antlr       |          | BSD-3-Clause | 0          | 0               |
| apiguardian-api | 1.1.2   | org.apiguardian |          | Apache-2.0   | 0          | 0               |
| asm             | 9.8     | org.ow2.asm     |          | BSD-3-Clause | 0          | 0               |
| asm-analysis    | 9.8     | org.ow2.asm     |          | BSD-3-Clause | 0          | 0               |
| asm-commons     | 9.8     | org.ow2.asm     |          | BSD-3-Clause | 0          | 0               |
| asm-tree        | 9.8     | org.ow2.asm     |          | BSD-3-Clause | 0          | 0               |
| asm-util        | 9.8     | org.ow2.asm     |          | BSD-3-Clause | 0          | 0               |
| bcel            | 6.10.0  | org.apache.bcel |          | Apache-2.0   | 0          | 0               |



## Выгрузка результатов

Выгружаем результаты в файл dtrack-findings.json

```
vladinir-kuryndin@vladinir-kuryndin-VMware-Virtual-Platform:~/dtrack$ sudo curl -s -H "X-API-Key: odt_pchp6V6x_6mIzaRhcqRdhJeZduP2K7iy8lsVkl37I" -o dtrack-findings.json "http://localhost:8081/api/v1/finding/project/29f006aa-019f-49fb-97a8-6df0f7aca576/export"
```

Данный файл будет добавлен к проекту при сдаче домашнего задания (см файл homework4.zip)

## Сравнительный анализ результатов

Результаты Dependency-Check (отчёт: dependency-check-report.html)

Dependency-Check обнаружил 2 уязвимости (2 CVE):

- CVE-2025-48734
- CVE-2025-48924

Отчёт представлен в виде HTML и удобен тем, что его можно открыть офлайн и приложить к отчёту как артефакт.

Результаты Dependency-Track (экспорт: dtrack-findings.json)

Dependency-Track по загруженному SBOM сформировал **4 finding / 4 CVE**:

- CVE-2025-48734
- CVE-2025-48924
- CVE-2025-53864
- CVE-2025-68161

Также в интерфейсе проекта отображается список компонентов (в моём случае импортировано около **80 компонентов** из SBOM), а анализ уязвимостей проводится на стороне платформы (см разделы выше).

## Уязвимости, найденные только одним инструментом

### Только Dependency-Track:

- CVE-2025-53864
- CVE-2025-68161

### Только Dependency-Check:

- отсутствуют (все CVE из Dependency-Check также присутствуют в Dependency-Track)

## Объяснение разницы между инструментами

Скорее всего, разница в количестве найденных уязвимостей обусловлена тем, что Dependency-Track был дополнительно настроен на анализ через Sonatype OSS Index (анализ по purl из SBOM), тогда как Dependency-Check выполнял локальное сканирование и сопоставление зависимостей со своими источниками (прежде всего NVD).

## Сравнение по критериям из задания

- 1) Количество найденных уязвимостей
  - Dependency-Check: **2**
  - Dependency-Track: **4**
- 2) Уровень детализации отчётов
  - Dependency-Check: даёт «build-time» отчёт, ориентированный на локальный скан проекта; удобно для CI — один HTML/JSON-отчёт, который можно сохранять артефактом.
  - Dependency-Track: даёт платформенный взгляд — привязка уязвимостей к компонентам (SBOM), отображение зависимостей/компонентов в UI, возможность выгрузки findings через API, а также последующая работа с результатами (аудит/триаж/история).
- 3) Удобство интерпретации
  - Dependency-Check: проще как “один файл отчёта”, его можно быстро приложить в отчёт/артефакты.
  - Dependency-Track: удобнее для навигации по компонентам и уязвимостям в веб-интерфейсе, особенно когда нужно смотреть зависимости как набор компонентов проекта.
- 4) Время выполнения
  - Локально, первый запуск:
    - Dependency-Check: **около 40 минут с учетом долгого импорта баз (без NVD key)** .
    - Dependency-Track: **импорт SBOM + анализ - тоже около 40 минут** , пришлось по логам выяснять, почему не сразу показывались компоненты и уязвимости.
  - На Github Actions время выполнения:
    - Dependency check 4 m 9s (с использованием NVD key)
    - На Dependency Track замерим время с помощью следующего скрипта:

TOKEN="dd9becb6-fe21-4a98-a149-00cca31ed7a0"

```
sudo docker logs --timestamps dtrack-apiserver-1 \
| grep "$TOKEN" \
| grep -i "BOM processed successfully" \
| tail -n 1
```

```
vladimir-kuryndin@vladimir-kuryndin-Virtual-Platform: /var/lib/docker/actions-runner$ TOKEN=dd9becb6-fe21-4a98-a149-00cca31ed7a0
sudo docker logs --timestamps dtrack-apiserver-1 \
| grep "$TOKEN" \
| grep -i "BOM processed successfully" \
| tail -n 1
2025-12-24T22:15:29.185755382Z 2025-12-24 22:15:29,185 INFO [BomUploadProcessingTask] BOM processed successfully in 00:00:00.176 [bomSerialNumber=85c57be7-d8b1-4947-b433-f115feb5c3ae, bomFormat=CycloneDX, bomUploadToken=dd9becb6-fe21-4a98-a149-00cca31ed7a0, projectName=shortlinkapp, bomSpecVersion=1.6, projectUuid=29f006aa-019f-49fb-97a8-6df0f7aca576, projectVersion=1.0-SNAPSHOT, bomVersion=1]
```

025-12-24T22:15:29.185755382Z 2025-12-24 22:15:29,185 INFO [BomUploadProcessingTask] BOM processed successfully in 00:00:00.176 [bomSerialNumber=85c57be7-d8b1-4947-b433-f115feb5c3ae, bomFormat=CycloneDX, bomUploadToken=dd9becb6-fe21-4a98-a149-00cca31ed7a0, projectName=shortlinkapp, bomSpecVersion=1.6, projectUuid=29f006aa-019f-49fb-97a8-6df0f7aca576, projectVersion=1.0-SNAPSHOT, bomVersion=1]

Соответственно, такая быстрая скорость (00:00:00.176) связана с тем, что все уже закешировано на сервере и что мы уже не первый раз прогоняем наш один и тот же BOM.

## Установка и настройка CI/CD на GitHub

Подключаем self hosted github runner

```
vladimir-kuryndin@vladimir-kuryndin-Virtual-Platform: /var/lib/docker/actions-runner$ cd /var/lib/docker/actions-runner
curl -o actions-runner-linux-x64-2.330.0.tar.gz -L \
https://github.com/actions/runner/releases/download/v2.330.0/actions-runner-linux-x64-2.330.0.tar.gz

echo "af5c33fa94f3cc33b8e97937939136a6b04197e6dadfcfb3b6e33ae1bf41e79a actions-runner-linux-x64-2.330.0.tar.gz" | sha256sum -c

tar xzf ./actions-runner-linux-x64-2.330.0.tar.gz
sudo ./bin/installdependencies.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 211M  100 211M    0     0 20.1M    0  0:00:10  0:00:10 --:--:-- 22.6M
actions-runner-linux-x64-2.330.0.tar.gz: OK
-----OS Information-----
PRETTY_NAME="Ubuntu 24.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.3 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

```
vladimir-kuryndin@vladimir-kuryndin-Virtual-Platform: /var/lib/docker/actions-runner$ ./config.sh --url https://github.com/vkuryndin/shortlinkapp --token B
XLSNFBY4VF3VJEPQZT63JJRSK6 --name ubuntu24-dtrack --labels self-hosted,linux,dtrack --work _work --unattended

-----
GitHub Actions
Self-hosted runner registration
-----

# Authentication

✓ Connected to GitHub

# Runner Registration

✓ Runner successfully added

# Runner settings

✓ Settings Saved.
```

```
vladimir-kuryndin@vladimir-kuryndin-Virtual-Platform: /var/lib/docker/actions-runner$ sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
Creating launch runner in /etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service
Run as uid: 1000
gid: 1000
Created symlink /etc/systemd/system/multi-user.target.wants/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service → /etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service.

/etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service
● actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service - GitHub Actions Runner (vkuryndin-shortlinkapp.ubuntu24-dtrack)
   Loaded: loaded (/etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-12-25 01:17:50 +04; 6ms ago
     Main PID: 53920 (runsv.sh)
       Tasks: 1 (limit: 9368)
      Memory: 688.0K (peak: 888.0K)
         CPU: 2ms
    CGroup: /system.slice/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service
            └─53920 /bin/bash /var/lib/docker/actions-runner/runsv.sh
              └─53923 /externals/node20/bin/node /bin/RunnerService.js

Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform systemd[1]: Started actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service - GitHub Action...dtrack).
Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform runsv.sh[53920]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/sbin
Hint: Some lines were ellipsized, use -l to show in full.

/etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service
● actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service - GitHub Actions Runner (vkuryndin-shortlinkapp.ubuntu24-dtrack)
   Loaded: loaded (/etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-12-25 01:17:50 +04; 78ms ago
     Main PID: 53920 (runsv.sh)
       Tasks: 9 (limit: 9368)
      Memory: 21.1M (peak: 21.1M)
         CPU: 49ms
    CGroup: /system.slice/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service
            └─53920 /bin/bash /var/lib/docker/actions-runner/runsv.sh
              └─53923 /externals/node20/bin/node /bin/RunnerService.js
                └─53937 /var/lib/docker/actions-runner/bin/Runner.Listener run --startuptype service

Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform systemd[1]: Started actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service - GitHub Action...dtrack).
Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform runsv.sh[53920]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/sbin
Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform runsv.sh[53923]: Starting Runner listener with startup type: service
Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform runsv.sh[53923]: Started listener process, pid: 53937
Dec 25 01:17:50 vladimir-kuryndin-Virtual-Platform runsv.sh[53923]: Started running service
Hint: Some lines were ellipsized, use -l to show in full.
```

Получаем готовый runner.

vkuryndin / shortlinkapp

Type to search

code

Issues

Pull requests

Actions

Projects

Wiki

Security

1

Insights

Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

General

Runners

Models

Webhooks

Copilot

Runners

Host your own runners and customize the environment used to run jobs in your GitHub Actions workflows. [Learn more about self-hosted runners.](#)

Runners

Status

ubuntu24-dtrack

self-hosted

Linux

X64

dtrack

Idle

New self-hosted runner

## Проверка работоспособности github self hosted runner

```

vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform: /var/lib/docker/actions-runner$ sudo systemctl status actions.runner.* | cat
● actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service - GitHub Actions Runner (vkuryndin-shortlinkapp.ubuntu24-dtrack)
   Loaded: loaded (/etc/systemd/system/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-12-25 01:17:50 +04; 9h ago
     Main PID: 53920 (runsv.sh)
        Tasks: 20 (limit: 9368)
      Memory: 118.7M (peak: 241.1M)
         CPU: 23.660s
    CGroup: /system.slice/actions.runner.vkuryndin-shortlinkapp.ubuntu24-dtrack.service
            └─53920 /bin/bash /var/lib/docker/actions-runner/runsv.sh
              └─53923 ./externals/node20/bin/node ./bin/RunnerService.js
                └─53937 /var/lib/docker/actions-runner/bin/Runner.Listener run --startuptype service

Dec 25 01:17:56 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 21:17:56Z: Running job: security (Upload SBOM to Dependency-Track)
Dec 25 01:18:09 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 21:18:09Z: Job security (Upload SBOM to Dependency-Track) completed with result: Failed
Dec 25 01:32:11 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 21:32:11Z: Running job: security (Upload SBOM to Dependency-Track)
Dec 25 01:32:31 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 21:32:31Z: Job security (Upload SBOM to Dependency-Track) completed with result: Succeeded
Dec 25 02:07:30 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 22:07:30Z: Running job: security (Upload SBOM to Dependency-Track)
Dec 25 02:07:50 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 22:07:50Z: Job security (Upload SBOM to Dependency-Track) completed with result: Succeeded
Dec 25 02:15:18 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 22:15:18Z: Running job: security (Upload SBOM to Dependency-Track)
Dec 25 02:15:41 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-24 22:15:41Z: Job security (Upload SBOM to Dependency-Track) completed with result: Succeeded
Dec 25 10:24:26 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-25 06:24:26Z: Running job: smoke
Dec 25 10:24:34 vladimir-kuryndin-VMware-Virtual-Platform runsv.sh[53923]: 2025-12-25 06:24:34Z: Job smoke completed with result: Succeeded

```

```

name: Runner smoke test

on:
  workflow_dispatch:

jobs:
  smoke:
    runs-on: [self-hosted, dtrack]
    steps:
      - name: Who am I
        run: |
          set -eux
          hostname
          whoami
          pwd
          uname -a
          df -h
          docker --version || true

```

[Runner smoke test · Workflow runs · vkuryndin/shortlinkapp](#)

← Runner smoke test  
✓ Runner smoke test #1

Re-run all jobs ...

Summary

All jobs

✓ smoke

Run details

Usage

Workflow file

smoke

succeeded 12 minutes ago in 8s

Search logs

Re-run all jobs

...

> ✓ Set up job

1s

✓ Who am I

0s

```
1 ▶ Run set -eux
10 + hostname
11 + whoami
12 + pwd
13 + uname -a
14 + df -h
15 vladimir-kuryndin-VWare-Virtual-Platform
16 vladimir-kuryndin
17 /var/lib/docker/actions-runner/_work/shortlinkapp/shortlinkapp
18 Linux vladimir-kuryndin-VWare-Virtual-Platform 6.14.0-37-generic #37-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Nov 20 10:25:38 UTC 2 x86_64 x86_64 x86_64
GNU/Linux
19 Filesystem      Size  Used Avail Use% Mounted on
20 tmpfs            790M  2.5M  787M   1% /run
21 /dev/sda2        20G   17G   1.9G   91% /
22 tmpfs            3.9G   0 3.9G   0% /dev/shm
23 tmpfs            5.0M  8.0K  5.0M   1% /run/lock
24 vmhgfs-fuse      461G  436G   26G  95% /mnt/hgfs
25 /dev/mapper/vg_raid5-lv_data 2.0G   40K   1.9G   1% /mnt/raid5
26 /dev/md1         1.7G   36K   1.6G   1% /mnt/data
27 tmpfs            790M  144K  790M   1% /run/user/1000
28 /dev/sr1         6.0G   6.0G   0 100% /media/vladimir-kuryndin/Ubuntu 24.04.3 LTS amd64
29 /dev/sdh1        20G   7.1G   12G  39% /var/lib/docker
30 + docker --version
31 Docker version 28.2.2, build 28.2.2-0ubuntu1-24.04.1
```

✓ Complete job

0s

## Настройка секретов GitHub

Настраиваем дополнительные переменные для работы selfhosted gitlab runner.

|                                                                                                    |              |                                                                                       |                                                                                       |
|----------------------------------------------------------------------------------------------------|--------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|  DTRACK_API_KEY | 1 minute ago |  |  |
|  DTRACK_URL     | now          |  |  |
|  NVD_API_KEY    | 6 hours ago  |  |  |

## Создаем файл CI/CD в Github Actions

Создадим файл security.yml

Прошу обратить внимание, что ручной запуск workflow\_dispatch оставлен для отладки! Я понимаю, что для задания должен быть только on: pull\_request.

```
name: SBOM + Dependency Scan
```

```
on:
```

```
  pull_request:
```

```
  workflow_dispatch:
```

```
    inputs:
```

```
      projectName:
```

```
        description: "Dependency-Track project name (default: repo name)"
```

```
        required: false
```

```
      projectVersion:
```

```
        description: "Dependency-Track project version (default: branch name)"
```

```
        required: false
```

```
      runDependencyCheck:
```



```

        description: "Run OWASP Dependency-Check (true/false)"
        required: false
        default: "true"
    uploadToDependencyTrack:
        description: "Upload SBOM to Dependency-Track (true/false)"
        required: false
        default: "true"

permissions:
  contents: read

env:
  SBOM_XML: build/reports/bom.xml

jobs:
  build_sbom:
    name: build (Generate SBOM)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
          cache: gradle

      - name: Grant execute permission for gradlew
        run: chmod +x ./gradlew

      - name: Generate CycloneDX SBOM
        run: ./gradlew --no-daemon cyclonedxBom

      - name: Verify SBOM exists
        run: |
          if [ ! -f "${SBOM_XML}" ]; then
            echo "SBOM not found at ${SBOM_XML}"
            echo "Listing build/reports:"
            find build/reports -maxdepth 3 -type f -print
            exit 1
          fi

      - name: Upload SBOM artifact
        uses: actions/upload-artifact@v4
        with:
          name: sbom-cyclonedx
          path: build/reports/bom.*
          retention-days: 7

  dependency_check:
    name: security (Dependency-Check)
    runs-on: ubuntu-latest
    needs: build_sbom
    if: ${{ github.event_name == 'pull_request' || github.event.inputs.runDependencyCheck == 'true' }}
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
          cache: gradle

      - name: Grant execute permission for gradlew
        run: chmod +x ./gradlew

```

```

- name: Run OWASP Dependency-Check
  env:
    NVD_API_KEY: ${ secrets.NVD_API_KEY }
  run: ./gradlew --no-daemon dependencyCheckAnalyze

- name: Upload Dependency-Check report
  uses: actions/upload-artifact@v4
  with:
    name: dependency-check-report
    path: build/reports/dependency-check/
    retention-days: 7

upload_to_dependencytrack:
  name: security (Upload SBOM to Dependency-Track)
  needs: build_sbom
  if: ${ github.event_name == 'pull_request' || github.event.inputs.uploadToDependencyTrack ==
'true' }}
  runs-on: [ self-hosted, dtrack ]
  steps:
    - uses: actions/checkout@v4

    - name: Download SBOM artifact
      uses: actions/download-artifact@v4
      with:
        name: sbom-cyclonedx
        path: build/reports

    - name: Upload SBOM and export findings from Dependency-Track
      env:
        DTRACK_URL: ${ secrets.DTRACK_URL }
        DTRACK_API_KEY: ${ secrets.DTRACK_API_KEY }
      run: |
        set -eou pipefail

        PROJECT_NAME_INPUT="${ github.event.inputs.projectName }"
        PROJECT_VERSION_INPUT="${ github.event.inputs.projectVersion }"

        if [ -n "${PROJECT_NAME_INPUT}" ]; then
          PROJECT_NAME="${PROJECT_NAME_INPUT}"
        else
          PROJECT_NAME="${ github.event.repository.name }"
        fi

        if [ -n "${PROJECT_VERSION_INPUT}" ]; then
          PROJECT_VERSION="${PROJECT_VERSION_INPUT}"
        else
          PROJECT_VERSION="${ github.head_ref || github.ref_name }"
        fi

        test -f "${SBOM_XML}"

        # 1) Upload BOM -> get processing token
        echo "Uploading BOM for ${PROJECT_NAME}@${PROJECT_VERSION} to ${DTRACK_URL}"
        TOKEN=$(curl -sS -X POST "${DTRACK_URL}/api/v1/bom" \
          -H "X-Api-Key: ${DTRACK_API_KEY}" \
          -F "projectName=${PROJECT_NAME}" \
          -F "projectVersion=${PROJECT_VERSION}" \
          -F "autoCreate=true" \
          -F "bom=@${SBOM_XML}" | python3 -c 'import sys,json;
print(json.load(sys.stdin)["token"])')

        echo "{\"token\":\"${TOKEN}\"}" > dtrack-upload.json
        echo "BOM token: ${TOKEN}"

        # 2) Wait until processing is done (max ~5 min)

```

```

for i in {1..60}; do
    STATUS="$(curl -sS -H "X-API-Key: ${DTRACK_API_KEY}" \
        "${DTRACK_URL}/api/v1/bom/token/${TOKEN}")"
    echo "token status: ${STATUS}"
    echo "${STATUS}" | grep -q '"processing":false' && break
    sleep 5
done

# 3) Lookup project UUID
PROJECT_UUID="$(curl -sS -G "${DTRACK_URL}/api/v1/project/lookup" \
    -H "X-API-Key: ${DTRACK_API_KEY}" \
    --data-urlencode "name=${PROJECT_NAME}" \
    --data-urlencode "version=${PROJECT_VERSION}" \
    | python3 -c 'import sys,json; print(json.load(sys.stdin)["uuid"])' )"

echo "Project UUID: ${PROJECT_UUID}"

# 4) Export findings + vulnerabilities (две разные выборки)
curl -sS -H "X-API-Key: ${DTRACK_API_KEY}" \
    "${DTRACK_URL}/api/v1/finding/project/${PROJECT_UUID}" \
    -o dtrack-findings.json

curl -sS -H "X-API-Key: ${DTRACK_API_KEY}" \
    "${DTRACK_URL}/api/v1/vulnerability/project/${PROJECT_UUID}" \
    -o dtrack-vulnerabilities.json

# Small sanity output
echo "findings bytes: $(stat -c%s dtrack-findings.json || true)"
echo "vulns bytes: $(stat -c%s dtrack-vulnerabilities.json || true)"

- name: Upload Dependency-Track results
  uses: actions/upload-artifact@v4
  with:
    name: dependency-track-results
    path: |
      dtrack-upload.json
      dtrack-findings.json
      dtrack-vulnerabilities.json
    retention-days: 7

```

# Результат запуска

Summary

All jobs

build (Generate SBOM)

security (Dependency-Check)

security (Upload SBOM to Dependency-Track)

Run details

Usage

Workflow file

Manually triggered 6 minutes ago

Status

Total duration

Artifacts

vkuryndin - 5baa83c master

Success

4m 40s

3

security.yml

on: workflow\_dispatch

build (Generate SBOM) 33s

security (Dependency-Ch... 4m 0s)

security (Upload SBOM to ... 20s)

Artifacts

Produced during runtime

| Name                    | Size      | Digest                                          |
|-------------------------|-----------|-------------------------------------------------|
| dependency-check-report | 175 KB    | sha256:adea4d5edf64ad7e4ac668d4bb02371dc36...   |
| dependency-track-upload | 263 Bytes | sha256:9321cb240f909de39ef29fcf3436825ef2f7d... |
| sbom-cyclonedx          | 345 KB    | sha256:266d30b99582006c88500f9b6141c34d76c11... |

Подробнее см. [SBOM + Dependency Scan · Workflow runs · vkuryndin/shortlinkapp](#)